

Windows Security Model Simulation

Access Tokens, Discretionary Access Control Lists & Privilege Checks



Subject: Operating Systems

Project Type: Simulation & Comparative Study

Language: C (Standard C11)

Repository: github.com/Sriram-Nambiar/os-security-simulator

Table of Contents

1. Introduction
2. Objectives
3. Background & Theory
4. System Architecture
5. ACE Evaluation Algorithm & Flowchart
6. Implementation Details
7. Program Output & Screenshots
8. Test Scenario Analysis
9. How to Compile & Run
10. Conclusion

1. Introduction

Security is one of the most critical subsystems of any modern operating system. Microsoft Windows employs a sophisticated **Discretionary Access Control (DAC)** model where every running process carries an *Access Token* describing its identity and privileges, and every protected resource (files, registry keys, services, kernel objects) is guarded by a *Security Descriptor* containing an ordered list of *Access Control Entries (ACEs)*.

This project implements a **console-based C simulator** that models the complete Windows NT security evaluation pipeline. The simulator creates users, groups, processes (with tokens), and resources (with DACLs), then demonstrates access being **GRANTED** or **DENIED** based on the same logical rules that the real Windows Security Reference Monitor (SRM) uses.

Key Distinction: This is not a toy demo — the ACE evaluation logic (deny-before-allow, ordered walk, implicit deny) directly mirrors the algorithm described in Microsoft's official documentation for the `SeAccessCheck()` kernel routine.

2. Objectives

1. Model **Users** and **Groups** as security principals with unique identifiers (SIDs).
2. Construct **Access Tokens** for each simulated process, containing User SID, Group SIDs, and Privileges.
3. Attach **Discretionary Access Control Lists (DACLs)** to simulated file resources, each containing an ordered sequence of ALLOW and DENY ACEs.
4. Implement the **Windows ACE evaluation algorithm** (owner check, explicit deny, allow accumulation, implicit deny).
5. Demonstrate **Privilege Bypass** (e.g., `SeBackupPrivilege`) overriding normal DACL checks.
6. Provide an **Interactive Mode** for users to modify DACLs and test custom access scenarios at runtime.

3. Background & Theory

3.1 What is an Access Token?

In the Windows NT security architecture, an **Access Token** is a kernel object generated at user logon. It encapsulates the complete security context of a process:

Token Component	Description	Example
User SID	Uniquely identifies the logged-in user account	S-1-5-21-...-1001 (simplified to UID 1 in our simulator)
Group SIDs	List of all security groups the user is a member of	Administrators, Users, Backup Operators
Privileges	System-wide capabilities that bypass normal access checks	SeBackupPrivilege , SeDebugPrivilege

Every process inherits the token of the user who launched it. Child processes inherit the parent's token.

3.2 What is a Security Descriptor & DACL?

Every securable object in Windows has a **Security Descriptor** that contains:

- **Owner SID:** The user who owns the object (has implicit full control).
- **DACL (Discretionary Access Control List):** An ordered sequence of Access Control Entries (ACEs) specifying who is allowed or denied access.

Each ACE in a DACL contains three fields:

ACE Field	Description
Type	ALLOW or DENY
Trustee (SID)	The user or group this ACE applies to

Rights Bitmask

READ , WRITE , EXECUTE , DELETE (or any combination)

3.3 Why Order Matters: Deny-Before-Allow

The fundamental rule of Windows DACL evaluation is that **ACEs are processed sequentially from first to last**. Microsoft recommends placing DENY ACEs before ALLOW ACEs. If a DENY ACE matches the requesting token and covers any requested right, access is immediately refused — the system does *not* continue checking subsequent ALLOW entries. This is a deliberate security design: explicit denials always take precedence.

3.4 What are Privileges?

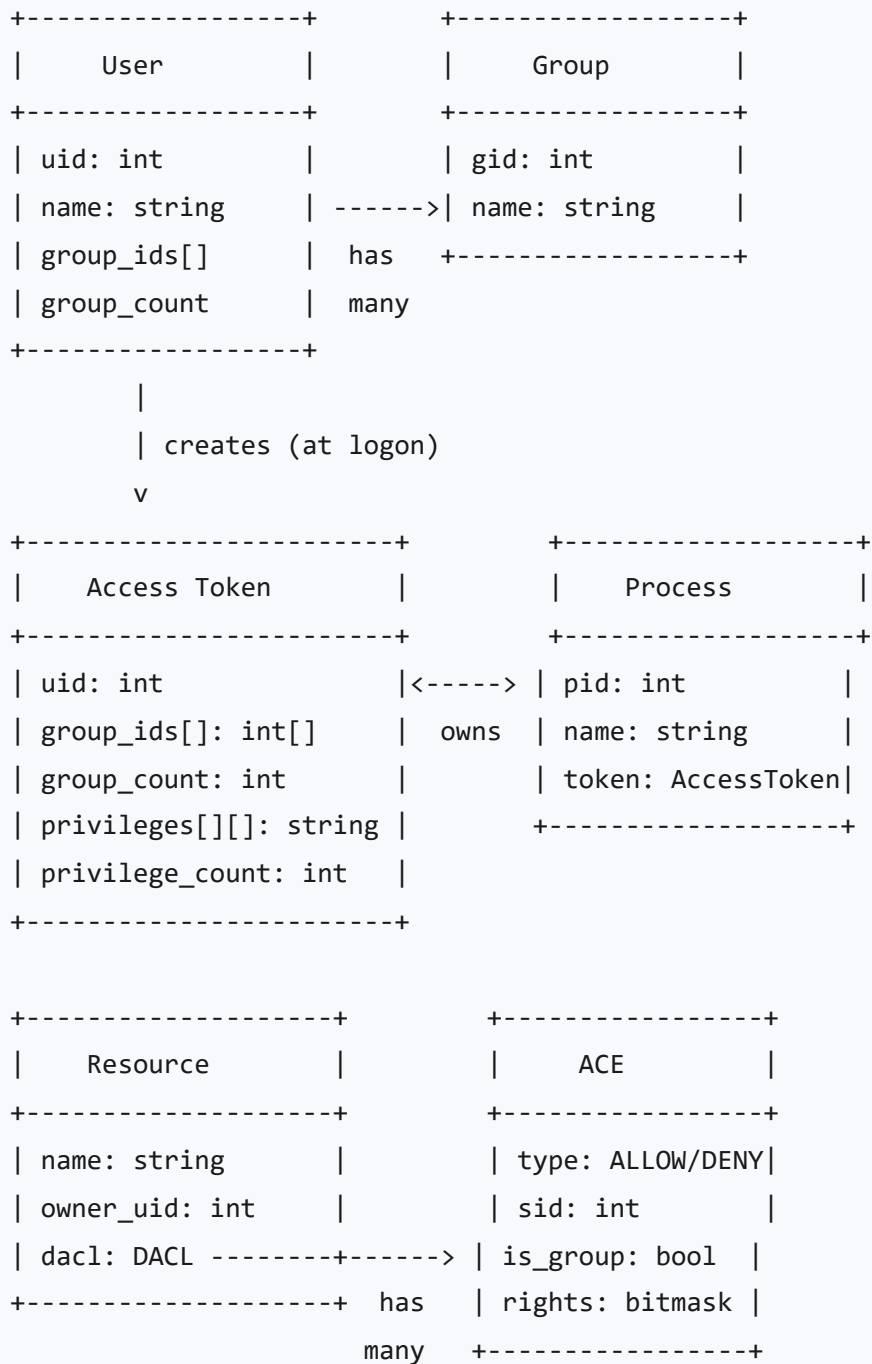
Privileges are system-wide overrides granted to specific accounts or groups. Unlike ACEs (which are per-object), privileges apply globally. For example:

- `SeBackupPrivilege` — Allows reading any file regardless of its DACL (used by backup software like Windows Backup, Veeam).
- `SeDebugPrivilege` — Allows attaching a debugger to any process.
- `SeTakeOwnershipPrivilege` — Allows taking ownership of any object.

4. System Architecture

4.1 Data Structure Diagram

The following diagram shows how the simulator's core data structures relate to each other:



4.2 Rights Bitmask Encoding

Right	Bit Value	Hex	Description
READ	1	0x01	Read contents of a file or resource
WRITE	2	0x02	Modify or write to a file

EXECUTE	4	0x04	Execute a program or script
DELETE	8	0x08	Delete a file or resource
FULL CONTROL	15	0x0F	All rights combined (R+W+X+D)

4.3 Simulated Environment

Users & Groups

UID	User	Groups	Role
1	Alice	Administrators, Users	System administrator
2	Bob	Users	Standard user
3	Charlie	Guests	Guest / restricted user
4	Dave	Users, Backup Operators	Backup technician

Processes & Tokens

PID	Process	User	Groups in Token	Privileges
1	cmd.exe	Alice	Administrators, Users	SeBackupPrivilege
2	notepad.exe	Bob	Users	(none)
3	explorer.exe	Charlie	Guests	(none)
4	backup.exe	Dave	Users, Backup Operators	SeBackupPrivilege

Resources & DACLs

Resource	Owner	ACE #1	ACE #2	ACE #3
C:\Confidential\report.docx	Alice	DENY Guests [FULL]	ALLOW Admins [FULL]	ALLOW Users [READ]
D:\Shared\project.xlsx	Bob	DENY Backup Ops	ALLOW Users	—

		[WRITE DELETE]	[READ WRITE]	
C:\System\boot.ini	Alice	ALLOW Admins [READ]	—	—

5. ACE Evaluation Algorithm & Flowchart

5.1 Algorithm (Pseudocode)

```
FUNCTION access_check(token, resource, requested_rights):

    // Step 0: Owner override
    IF token.uid == resource.owner_uid THEN
        RETURN GRANTED ("Owner has implicit Full Control")

    // Step 1: Walk DACL entries sequentially
    granted_mask = 0x00

    FOR EACH ace IN resource.dacl.entries:
        IF ace does NOT match token (user SID or any group SID):
            CONTINUE to next ACE

        IF ace.type == DENY:
            IF (ace.rights AND requested_rights) != 0:
                RETURN DENIED ("Explicit DENY matched")

        ELSE IF ace.type == ALLOW:
            granted_mask = granted_mask OR ace.rights

    // Step 2: Check accumulated rights
    IF (granted_mask AND requested_rights) == requested_rights:
        RETURN GRANTED ("Sufficient ALLOW ACEs accumulated")

    // Step 3: Implicit deny (default)
    RETURN DENIED ("No ACE grants remaining rights")
```

5.2 Flowchart

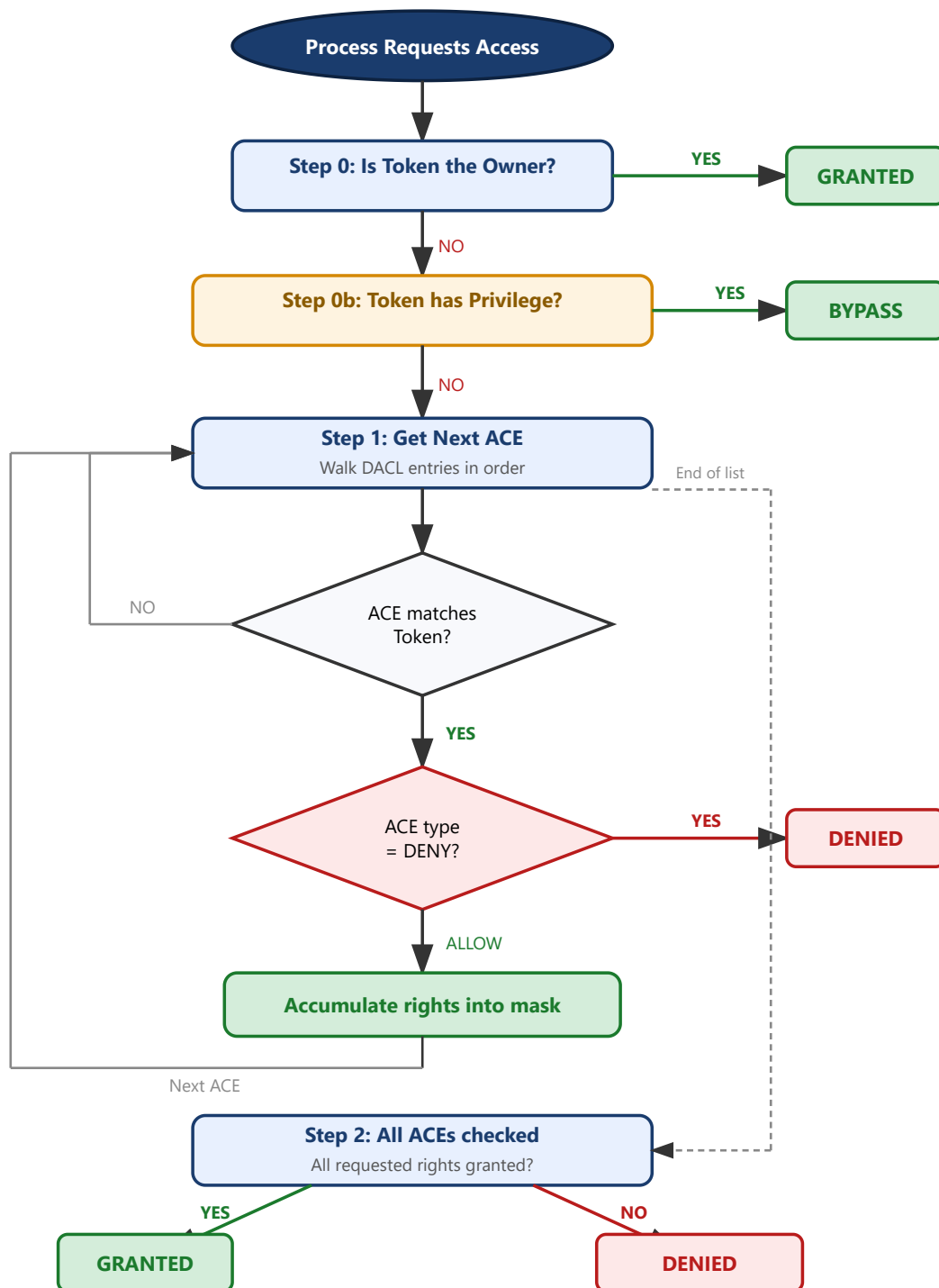


Figure 1: Windows DACL Access Check Evaluation Flowchart

6. Implementation Details

6.1 Key Functions

Function	Purpose
<code>access_check()</code>	Core DACL evaluation: owner check, deny-before-allow walk, implicit deny
<code>ace_matches_token()</code>	Determines if an ACE's trustee SID matches the requesting token (user SID or any group SID)
<code>token_has_privilege()</code>	Checks whether a token carries a specific named privilege string
<code>build_demo_world()</code>	Constructs all users, groups, resources, processes, and DACLs for the preset demo
<code>run_demo()</code>	Executes all 8 preset access-check scenarios with formatted output
<code>interactive_mode()</code>	Menu-driven loop allowing runtime inspection and modification of tokens and DACLs
<code>read_int_safe()</code>	Robust input reader using <code>fgets</code> + <code>sscanf</code> to prevent infinite loops on invalid input
<code>rights_to_string()</code>	Converts a bitmask to a human-readable string (e.g., "READ WRITE")

6.2 Core Access Check Logic (C Code)

```
int access_check(const AccessToken *token, const Resource *res,
                int requested_rights, char *reason, int reason_size)
{
    // Step 0: Owner always gets implicit full control
    if (token->uid == res->owner_uid) {
        snprintf(reason, reason_size,
                 "GRANTED -- requestor is the OWNER");
    }
}
```

```
        return 1;
    }

    // Step 1: Walk ACEs in order
    int granted_mask = 0;
    for (int i = 0; i < dacl->count; i++) {
        const ACE *ace = &dacl->entries[i];
        if (!ace_matches_token(ace, token)) continue;

        if (ace->type == ACE_DENY) {
            if (ace->rights & requested_rights)
                return 0; // DENIED immediately
        } else {
            granted_mask |= ace->rights; // accumulate
        }
    }

    // Step 2: Check accumulated rights
    if ((granted_mask & requested_rights) == requested_rights)
        return 1; // GRANTED

    // Step 3: Implicit deny
    return 0; // DENIED
}
```

6.3 Program Modes

Mode	How to Select	Description
Preset Demo	Press 1 at startup	Runs 8 curated scenarios automatically, showing environment setup and access check results
Interactive Mode	Press 2 at startup	Menu-driven interface to list users/tokens/DACLs, add custom ACEs, and perform arbitrary access checks

7. Program Output & Screenshots

7.1 Main Menu & Interactive Mode

When the program starts, the user selects between the preset demo (option 1) or interactive mode (option 2). The interactive mode provides a menu to inspect all security objects and perform custom access checks.

```
+-----+
| WINDOWS SECURITY MODEL SIMULATION |
| Access Tokens, DACLS & Privilege Checks |
+-----+

1. Run PRESET DEMO (recommended -- 8 scenarios)
2. INTERACTIVE mode (inspect/modify then test)
Enter choice (1 or 2): 2

-----
INTERACTIVE MODE MENU
-----

1. List all users
2. List all resources & DACLS
3. List all processes & tokens
4. Perform an access check
5. Add a new ACE to a resource's DACL
6. Return to main menu
Enter choice (1-6): █
```

Figure 2: Program launch showing main menu and interactive mode options

7.2 Users, Resources & DACL Configuration

The demo environment displays all configured users with their group memberships, followed by each resource's DACL showing the ordered ACE entries. Note how `report.docx` has a DENY ACE for Guests placed *before* the ALLOW ACEs — this is the critical deny-before-allow ordering.

```

USERS:
  UID 1  Alice      Groups: Administrators, Users
  UID 2  Bob        Groups: Users
  UID 3  Charlie    Groups: Guests
  UID 4  Dave       Groups: Users, Backup Operators

RESOURCES:
Resource : "C:\Confidential\report.docx"  Owner: Alice
DACL (3 ACEs):
  ACE #1: DENY   Guests      [READ WRITE EXECUTE DELETE]
  ACE #2: ALLOW  Administrators [READ WRITE EXECUTE DELETE]
  ACE #3: ALLOW  Users       [READ]

Resource : "D:\Shared\project.xlsx"  Owner: Bob
DACL (2 ACEs):
  ACE #1: DENY   Backup Operators [WRITE DELETE]
  ACE #2: ALLOW  Users            [READ WRITE]

```

Figure 3: Users, groups, resources, and DACL configurations

7.3 Access Check Scenario Results

The simulator evaluates each scenario and prints the reasoning trace: which token requested which rights on which resource, and why the result is GRANTED or DENIED.

ACCESS CHECK SCENARIOS

```

SCENARIO 1: Alice (Admin) reads report.docx
Process   : PID 1 (cmd.exe)
Resource  : C:\Confidential\report.docx
Requested: [READ]
>> GRANTED -- requestor is the OWNER (implicit Full Control)
[GRANTED] RESULT: ACCESS GRANTED

```

```

SCENARIO 2: Bob (User) reads report.docx
Process   : PID 2 (notepad.exe)
Resource  : C:\Confidential\report.docx
Requested: [READ]
>> GRANTED -- sufficient ALLOW ACEs found in DACL
[GRANTED] RESULT: ACCESS GRANTED

```

Figure 4: Access check scenarios showing GRANTED and DENIED results with explanations

8. Test Scenario Analysis

The preset demo runs 8 distinct scenarios, each designed to demonstrate a specific Windows security principle:

#	Scenario	Request	Result	Principle Demonstrated
1	Alice reads report.docx	READ	GRANTED	Owner Authority: File owner has implicit full control; DACL is not even consulted.
2	Bob reads report.docx	READ	GRANTED	Group Allow: ACE #3 grants READ to the Users group; Bob is a member of Users.
3	Bob writes report.docx	WRITE	DENIED	Implicit Deny: No ACE grants WRITE to Users; after walking all ACEs, the right remains unsatisfied.
4	Charlie reads report.docx	READ	DENIED	Explicit Deny Precedence: ACE #1 explicitly denies Guests all rights; even though Users are allowed READ, Guests are blocked first.
5	Dave writes project.xlsx	WRITE	DENIED	Deny-Before-Allow: Dave is in both Users (ALLOW WRITE) and Backup Operators (DENY WRITE). The DENY ACE appears first in the DACL, so access is immediately rejected.
6	Dave reads project.xlsx	READ	GRANTED	Targeted Deny: The DENY ACE only covers WRITE and DELETE; READ is not denied, so it accumulates from the Users ALLOW ACE.
7	Bob reads boot.ini	READ	DENIED	Implicit Deny: The only ACE grants READ to Administrators; Bob is not an admin, so the right is never granted.

8	Dave reads boot.ini	READ	GRANTED	Privilege Bypass: Dave's token carries SeBackupPrivilege , which overrides the DACL entirely — just like the real Windows Backup utility.
---	------------------------	------	---------	--

9. How to Compile & Run

9.1 Prerequisites

- A C compiler such as **GCC** (MinGW on Windows, Clang on macOS, or GCC on Linux).

9.2 Compilation

```
gcc -o windows_acl.exe windows_acl.c -Wall
```

9.3 Execution

```
.\windows_acl.exe
```

Select option **1** for the preset demo or **2** for interactive mode.

9.4 Source Repository

GitHub: <https://github.com/Sriram-Nambiar/os-security-simulator>

10. Conclusion

This project successfully demonstrates the core Windows NT security authorization model through a working C simulation. The key takeaways are:

1. **Every process has an identity** — Access Tokens carry the user SID, group SIDs, and privileges that collectively define what a process is allowed to do.
2. **Every resource has a policy** — DACLs define who can access what, using an ordered list of ALLOW and DENY ACEs.

3. **Order matters** — DENY ACEs evaluated before ALLOW ACEs ensure that explicit restrictions always take priority, even when a user belongs to multiple groups with conflicting permissions.
4. **Implicit deny is the default** — If no ACE explicitly grants a requested right, access is denied. This is a secure-by-default design philosophy.
5. **Privileges provide system-level overrides** — Special capabilities like `SeBackupPrivilege` allow trusted system utilities to bypass normal ACL restrictions for legitimate administrative tasks.

The simulator provides both a preset demonstration for quick evaluation and an interactive mode for experimentation, making it a practical educational tool for understanding operating system security fundamentals.

Windows Security Model Simulation — OS Security Simulator Project